

PC calificada - Desarrollo de preguntas

Explicación técnica paso a paso de las soluciones

Alcance	Se resolvieron las preguntas visibles en las capturas: 1, 2, 4, 5, 6, 8, 9 y 10. Las preguntas 3 y 7 no aparecen en las imágenes enviadas.
---------	--

Base usada: capturas de la PC calificada, diapositivas de Digital Certification / Mathematical Digital Signature / RSA Algorithm y código Java del módulo PKI del repositorio enviado.

Nota: las respuestas están ajustadas a la implementación del laboratorio en Java y Bouncy Castle.

Desarrollo detallado

La explicación se centra en lo que evalúa cada pregunta: estructura de CSR, certificado X.509, firma RSA, clases de Bouncy Castle y almacenamiento PKCS#12 mediante Java KeyStore.

Pregunta 1

Al procesar un CSR recibido, ¿qué datos críticos pertenecientes a la identidad y estructura del solicitante debe extraer la CA de forma obligatoria para mapearlos dentro del certificado final?

Respuesta correcta: A y C

- Un CSR PKCS#10 no es el certificado final. Es una solicitud que trae datos que la CA usará para crear el X.509.
- Los dos datos que sí se mapean directamente al certificado final son: Subject DN y llave pública del solicitante.
- En el código del laboratorio se observa esa extracción: el certificado se construye con `csr.getSubject()` y con la llave pública obtenida del CSR.
- La firma del CSR sirve para demostrar posesión de la llave privada asociada a la llave pública, pero esa firma original no se copia como campo del certificado final.

Fragmento de código relacionado:

```
JcaPKCS10CertificationRequest jcaCsr = new JcaPKCS10CertificationRequest(csr);
PublicKey userPublicKey = jcaCsr.getPublicKey();
```

```
X509v3CertificateBuilder cerBuilder =
    new JcaX509v3CertificateBuilder(
        issuerDN, serialNumber, startDate, endDate,
        csr.getSubject(), userPublicKey);
```

Por qué no van las demás opciones:

- B es falsa: el historial de certificados o revocaciones no se extrae del CSR para formar el certificado final.
- D es falsa: el algoritmo de cifrado de una base de datos local no pertenece a la identidad X.509 del solicitante.
- E es falsa como opción de mapeo: la firma del CSR se verifica, pero no se inserta como dato estructural del certificado emitido.

Pregunta 2

Al recuperar material criptográfico de un archivo PKCS12 mediante la API de Java, ¿qué excepción se lanza específicamente si el dato de activación es incorrecto?

Respuesta correcta: E

- El archivo PKCS#12 almacena material criptográfico protegido por un dato de activación, normalmente un password.
- Cuando se intenta recuperar la llave privada con el dato incorrecto, la API no puede reconstruir la llave protegida.
- Por eso, en la recuperación de la llave mediante `KeyStore.getKey(alias, password)`, la excepción específica es `UnrecoverableKeyException`.

Fragmento de código relacionado:

```
PrivateKey privateKey = (PrivateKey) ks.getKey(alias, password.toCharArray());
...
} catch (UnrecoverableKeyException ex) {
    System.getLogger(KeyStoreStorageService.class.getName())
        .log(System.Logger.Level.ERROR, (String) null, ex);
}
```

Nota técnica: si el password incorrecto falla durante `KeyStore.load(...)`, Java puede envolverlo como `IOException`. En esta pregunta, por las opciones disponibles y por la recuperación de la llave, la respuesta específica esperada es `UnrecoverableKeyException`.

Por qué no van las demás opciones:

- `FileNotFoundException` indica que el archivo no existe, no que el password sea incorrecto.
- `CertificateException` se relaciona con certificados, no con recuperación de una llave privada protegida.
- `KeyStoreException` es un problema general del contenedor, no específicamente del password de la llave.
- `NoSuchProviderException` indica ausencia del proveedor criptográfico, no error del dato de activación.

Pregunta 4

De acuerdo con la implementación matemática expuesta del protocolo firma digital con el algoritmo RSA, ¿qué componentes estructurales desde las llaves asimétricas son estrictamente necesarios para que el emisor y el receptor completen el ciclo de firma y verificación?

Respuesta correcta: C, D y E

- En RSA se trabaja módulo n , donde $n = p \cdot q$. Ese mismo n forma parte de la llave pública y de la llave privada.
- Para firmar se toma primero el resumen criptográfico del mensaje: $h = H(m)$.
- El emisor calcula el bloque de firma como $s = h^d \bmod n$. Aquí d es el exponente privado.
- El receptor recupera el valor firmado usando la llave pública: $h_{\text{rec}} = s^e \bmod n$. Aquí e es el exponente público.
- Finalmente compara h_{rec} contra $H(m)$. Si coinciden, la firma es válida.

Por qué no van las demás opciones:

- A no es necesaria durante el ciclo firma-verificación. El generador aleatorio interviene antes, al generar el par de llaves.
- B tampoco pertenece a la matemática RSA. La contraseña del contenedor solo protege almacenamiento local.

Pregunta 5

¿Qué clase de la biblioteca Bouncy Castle se encarga específicamente de empaquetar y vincular un Subject DN con una llave pública antes de aplicar la firma del CSR?

Respuesta correcta: D

- X500Name construye el nombre distinguido del sujeto, pero por sí solo no arma el CSR.
- La clase que vincula el Subject DN con la llave pública para formar la estructura PKCS#10 es JcaPKCS10CertificationRequestBuilder.
- Después de preparar esa estructura, se aplica la firma usando un ContentSigner creado con JcaContentSignerBuilder.

Fragmento de código relacionado:

```
X500Name subjectDN = new X500Name(userDN);

JcaPKCS10CertificationRequestBuilder csrBuilder =
    new JcaPKCS10CertificationRequestBuilder(
        subjectDN, userKeyPair.getPublic());

ContentSigner userSigner =
    new JcaContentSignerBuilder(RSA_SIGN_ALGORITHM)
        .setProvider(BCFIPS_PROVIDER)
        .build(userKeyPair.getPrivate());

PKCS10CertificationRequest csr = csrBuilder.build(userSigner);
```

Por qué no van las demás opciones:

- OperatorCreationException es una excepción, no una clase constructora del CSR.
- X500Name representa el DN, pero no lo vincula con la llave pública en una solicitud PKCS#10.
- JcaContentSignerBuilder crea el firmador, pero no empaqueta DN + llave pública.
- KeyPairGenerator genera llaves, no construye solicitudes CSR.

Pregunta 6

En el protocolo de firma digital con el algoritmo RSA, ¿qué operación matemática realiza estrictamente el emisor para generar el bloque numérico que representa la firma digital?

Respuesta correcta: E

- Una firma digital RSA no debe aplicarse directamente al mensaje completo en texto plano.
- Primero se calcula el hash del mensaje: $h = H(m)$.
- Luego el emisor usa su llave privada para generar la firma: $s = h^d \bmod n$.
- El receptor usará la llave pública para verificar: $h_{rec} = s^e \bmod n$, y compara con $H(m)$.

Por qué no van las demás opciones:

- A describe una operación con exponente público; eso corresponde a la verificación, no a la generación de firma.
- B falla porque habla del mensaje completo y no del hash.
- C y D no corresponden a RSA.

Pregunta 8

¿Qué componentes e información conjunta son estrictamente necesarios proveer al invocar el método `setKeyEntry` para guardar exitosamente una entidad de identidad digital en un `KeyStore` de tipo `PKCS12`?

Respuesta correcta: A, C y D

- En el laboratorio se guarda una identidad digital en un `KeyStore PKCS#12` con `setKeyEntry`.
- La llamada requiere un alias, la llave privada, un dato de activación para proteger la entrada y una cadena de certificados.
- Entre las opciones dadas aparecen alias, llave privada y cadena de certificación. El password también es necesario en la firma del método, pero no aparece como alternativa.
- La llave pública no se pasa de manera aislada porque está contenida dentro del certificado X.509 de la cadena.

Fragmento de código relacionado:

```
Certificate[] chain = new Certificate[]{certificate};
ks.setKeyEntry(alias, privateKey, password.toCharArray(), chain);
```

Por qué no van las demás opciones:

- B no se marca: `setKeyEntry` no recibe una llave pública aislada en bytes puros en esta implementación.
- E no se marca: la CRL sirve para verificar revocación, no para guardar la identidad digital en el `KeyStore`.

Pregunta 9

De acuerdo con los métodos implementados para almacenar `KeyStore's` y `TrustStore's` en contenedores `PKCS#12`, seleccione el procedimiento común entre ambos métodos.

Respuesta correcta: E

- El `KeyStore` y el `TrustStore` se almacenan como contenedores `PKCS#12`.
- En ambos métodos se recibe un password y, al escribir el archivo, se llama `store(..., password.toCharArray())`.
- Por eso el procedimiento común evaluado es establecer datos de activación para proteger el contenedor persistente.
- No se marca C porque el `KeyStore` almacena una entrada de llave privada con cadena de certificados, mientras que el `TrustStore` almacena un certificado confiable mediante `setCertificateEntry`.

Fragmento de código relacionado:

```
// KeyStore
ks.store(fos, password.toCharArray());
```

```
// TrustStore
ts.store(fos, password.toCharArray());
```

Si el cuestionario estuviera evaluando solo la línea de inicialización `load(null, null)`, A sería la otra observación técnica visible en el código. Para la pregunta tal como está formulada sobre almacenamiento `PKCS#12`, la solución más consistente es E.

Por qué no van las demás opciones:

- B es falsa: en el código, `saveToPKCS12File` usa `BCFIPS_PROVIDER`, mientras `saveToTrustStore` usa `KeyStore.getInstance(PKCS12_KEYSTORE_TYPE)` sin proveedor explícito.
- C es falsa: `TrustStore` no almacena una cadena de certificados con `setKeyEntry`, sino un certificado con `setCertificateEntry`.
- D es falsa: las listas de excepciones no son iguales; por ejemplo, el método de `KeyStore` maneja `NoSuchProviderException`.
- A aparece como paso de inicialización en memoria con `load(null, null)`, pero no representa la protección final del archivo `PKCS#12`. Para el almacenamiento final, lo común es el dato de activación.

Pregunta 10

En un certificado digital autofirmado, ¿qué relación matemática y formal existe entre el emisor (Issuer) y el sujeto (Subject)?

Respuesta correcta: B

- Un certificado autofirmado no es emitido por una CA externa.
- La misma entidad que aparece como Subject aparece también como Issuer.
- Formalmente, Issuer DN = Subject DN.
- La firma del certificado se genera con la llave privada de esa misma entidad y se verifica con su propia llave pública.

Fragmento de código relacionado:

```
X500Name dnName = new X500Name(caDn); // issuer = subject
```

```
X509v3CertificateBuilder cerBuilder =  
    new JcaX509v3CertificateBuilder(  
        dnName, serialNumber, startDate, endDate,  
        dnName, caKeyPair.getPublic());
```

Por qué no van las demás opciones:

- A no es una condición obligatoria de un certificado autofirmado.
- C confunde certificado con cifrado simétrico AES.
- D es falsa: Issuer no queda en null; coincide con Subject.
- E describe una CA intermedia externa, lo contrario de un certificado autofirmado.

Mapa conceptual rápido para repasar

Concepto	Idea clave
CSR PKCS#10	Trae el Subject DN y la llave pública del solicitante; se firma para demostrar posesión de la llave privada.
Certificado X.509	Contiene datos del sujeto, llave pública, datos del emisor, validez, serial y firma de la CA.
Firma RSA	Se calcula sobre el hash: $\text{firma} = H(m)^d \bmod n$; verificación = $\text{firma}^e \bmod n$.
Bouncy Castle CSR	JcaPKCS10CertificationRequestBuilder empaqueta Subject DN + llave pública; JcaContentSignerBuilder crea el firmador.
PKCS#12 / KeyStore	setKeyEntry guarda alias + llave privada + password + cadena de certificados.
TrustStore	Guarda certificados confiables; no guarda la llave privada de la entidad.
Certificado autofirmado	Issuer DN y Subject DN coinciden; se valida con su propia llave pública.